

+

新建

📁

電腦

🔍

自動化

📦

產出物

🕒

自訂

專案

沒有專案

工作階段

RAG ~ 數位員工 我的開發環境 ...

這是我目前的程式碼 和 完整規則

我的開發環境是AMD Ryzen AI M

我的開發環境是AMD Ryzen AI M

我的開發環境是AMD Ryzen AI M

電動自行車速度性能

如何使用Hermes 建立一個操作外

96g 顯卡 能佈署 那些 HERMES能

這是什麼植物

👤 廖昶哲

免費方案

🔍 搜尋

🔔 通知

📄 文件

👤 回答

🌐 連結

🖼️ 圖片

...

🔖 分享

RAG ~ 數位員工

我的開發環境是AMD Ryzen AI Max+ 395 (128GB)的Windows11上有LM Studio 可以運行Qwn3.6-35B-A3B + LINUX(VM) 的 Hermes Agent 運行的環境 我要如何在此基礎上實作出 一個專屬領域的諮詢專家如 產品客服/特定領域法律諮詢/...

該系統已在特定可擴充資料來源的知識庫內進行搜尋與回復

閱讀更多

研究完成 2 秒

基於你的硬體與現有環境 ( Windows 11 + LM Studio 跑 Qwen3.6-35B-A3B , 以及 Linux VM 內的 Hermes Agent ) , 你可以用「本地 RAG ( Retrieval-Augmented Generation ) 」實作一個專屬領域的諮詢專家 ( 如產品客服、法律諮詢等 ) , 讓系統只在你可擴充的知識庫內搜尋與回答。github +2

以下是 在 VM 內「小白也能實作」的完整步驟, 以 Hermes Agent 內建/官方推薦的 RAG 方案為主, 並尽量使用本地、可離線運作的元件。github +2

整體架構 ( 概念圖 )

知識庫來源: PDF/DOCX/TXT/Markdown 文件, 放在特定資料夾 ( 可擴充 ) 。

索引與嵌入: 在 VM 內 已啟用進階搜尋的免費預覽。瞭解詳情 文件切塊、做嵌入、建索引。agentupdate +1

詢問後續問題

+ 🔍 搜尋 📁 電腦

模型 電腦

前置條件檢查 ( 在 VM 內 )

1. 確認 Hermes Agent 已安裝且可正常對話

若尚未安裝, 依官方指令安裝 ( VM 內執行 ) :

bash

curl -fsSL [https://hermes-agent.nousresearch.com/install.sh](https://hermes-agent.nousresearch.com/install.sh) | bash

確認 hermes 指令可用, 且能與某個模型提供者對話 ( 例如先接 Ollama / OpenAI-compatible 端點 ) 。hermes-agent.nousresearch +2

2. 確認 Python 與 pip 環境

Hermes 的 RAG 依賴 Python 環境 ( 官方建議用 uv / venv ) 。若你已用官方 install.sh , 通常已幫你設定好。hermes-agent.nousresearch +1

3. 確認磁碟空間

嵌入模型 + 向量索引會吃空間, 建議至少保留 10 GB 給知識庫與模型。fast

步驟 1 : 選擇 RAG 方案 ( 從前兩種 )

詢問後續問題

+ 🔍 搜尋 📁 電腦

模型 電腦

週日: 單一領域知識庫 ( 例如「產品客服」或「呆頭法律條文」) 。

方案 B : Hermes + qmd skill ( 混合 BM25 + 向量 + rerank , 進階但更強 ) fast

優點: 檢索品質更好, 支援多集合 ( collection ) , 適合未來擴充多個領域。

適合: 之後想擴充多個「數位員工」( 客服、法務、技術支援等 ) 。

若你是第一次實作, 建議先走 方案 A ( ChromaDB ) , 熟悉流程後再升級到 qmd 。

步驟 2 : 安裝 RAG 所需套件 ( 以 ChromaDB 為例 )

在 VM 的 Hermes 環境中執行:

bash

pip install chromadb

# 若 Hermes 有獨立的 venv, 請確保是在該環境下執行

# 例如: source ~/.hermes/venv/bin/activate 後再 pip install

若你之後想用 PDF/DOCX 可額外安裝。

bash

詢問後續問題

+ 🔍 搜尋 📁 電腦

模型 電腦

步驟 3：設定 Hermes 使用 ChromaDB 作為知識庫

在 VM 終端機執行：

```
bash

hermes config set knowledge_base_provider chromadb
hermes config set knowledge_base_path ./hermes_knowledge_base
```

- knowledge\_base\_path 是你指定的資料夾，ChromaDB 會在此儲存向量索引。
- 之後你可以用相對或絕對路徑，例如： /home/youruser/hermes\_kb

步驟 4：準備你的領域知識文件

1. 在 VM 內建立一個資料夾，例如：

```
bash

mkdir -p ~/kb_product_support
```

2. 放入你的領域文件，例如：

- 產品手冊 (PDF)

詢問後續問題

+

搜尋

電腦

模型

↓

↑

text

~/kb\_product\_support/

- faq.md
- product\_manual.pdf
- policy\_2025.docx

之後若要擴充，只要把新文件丟進這個資料夾，再重新索引即可。

步驟 5：將文件加入 Hermes 知識庫並建立索引

使用 Hermes 的文件指令 ( 依官方範例 )：

```
bash

# 進入你想存放知識庫的目錄 (可選，只是方便管理)
cd ~

# 將整個資料夾加入知識庫
hermes docs add ~/kb product support --name product support kb
```

若你的版本指令略有不同

詢問後續問題

+

搜尋

電腦

模型

↓

↑

加入後，Hermes 會自動或手動觸發嵌入與索引建立。若需手動觸發，可執行：

```
bash

hermes docs index
```

( 若你的版本沒有 index 子指令，通常 docs add 時就會自動處理。 )

此時，ChromaDB 會在 ~/hermes\_knowledge\_base 中建立向量索引。

步驟 6：設定 LLM 端點 ( 接你的 LM Studio / Qwen )

你有兩個選擇：

選項 1：在 VM 內跑 Ollama / vLLM，再讓 Hermes 呼叫

- 優點：全部在 VM 內，架構單純。
- 缺點：要再安裝一個本地 LLM server。

選項 2：讓 VM 內的 Hermes 呼叫 Windows 上的 LM Studio API ( 推薦，充分利用你現有資源 )

1. 在 Windows 上啟用 LM Studio
  - 載入 Qwen3.6-35B-A3B

詢問後續問題

+

搜尋

電腦

模型

↓

↑

2. 在 VM 內測試可否連到 LM Studio

在 VM 終端機：

```
bash
```

```
curl http://<windows-ip>:1234/v1/models
```

若看得到模型列表，代表連線成功。

3. 在 Hermes 中設定模型提供者為 OpenAI-compatible

依 Hermes 文件設定 provider (不同版本指令可能略有差異，一般透過 hermes config 或編輯 ~/.hermes/config.yaml )：  
🔗 hermes-agent.nousresearch +1 範例 (YAML 設定檔)：

```
text

provider: openai
api_base: http://<windows-ip>:1234/v1
api_key: not-needed
model: qwen3.6-35b-a3b
```

或透過指令 (若支援)：

```
bash
hermes config set provider openai
```

詢問後續問題

+ 🔍 搜尋 📁 電腦

模型 📄 ⬇️ ⬆️

步驟 7：設定「領域專家」人設與限制 (系統提示詞)

為了讓系統只在知識庫內回答，你需要：

1. 定義系統提示詞 (system prompt)，例如：

```
text

你是一個「產品客服專家」，只能根據提供的知識庫內容回答問題。
如果知識庫中找不到足夠資訊，請明確告知「目前知識庫中沒有相關資料」，不要自行推測或編造答案。
回答時盡量引用知識庫中的條文或段落 (可用簡短說明來源文件名稱)。
```

2. 在 Hermes 中設定系統提示詞 (依版本不同，可用 hermes config 或編輯 config.yaml)：

🔗 hermes-agent.nousresearch 範例 (YAML)：

```
text

system_message: |
  你是一個「產品客服專家」，只能根據提供的知識庫內容回答問題。
  如果知識庫中找不到足夠資訊，請明確告知「目前知識庫中沒有相關資料」，不要自行推測或編造答案。
  回答時盡量引用知識庫中的條文或段落 (可用簡短說明來源文件名稱)。
```

詢問後續問題

+ 🔍 搜尋 📁 電腦

模型 📄 ⬇️ ⬆️

📄 知識庫：知識庫文件列表目前使用 1 個文件

在 VM 內執行：

```
bash
hermes chat
```

然後輸入與你的領域相關問題，例如：

- 「產品 A 的保固期限是多久？」
- 「根據內部規範，退貨流程是什麼？」

觀察：

- 回答是否基於你放入的文件內容。
- 若問與知識庫無關的問題，是否會說「知識庫中沒有相關資料」。

若效果不理想，可調整：

- 文件切塊大小 (若 Hermes 提供相關設定)。
- 系統提示詞的嚴格程度。
- 檢索回的文件數量 (↑ ↓)。

詢問後續問題

+ 🔍 搜尋 📁 電腦

模型 📄 ⬇️ ⬆️

- kb\_product\_support
- kb\_legal\_consult
- kb\_internal\_policy

每個集合對應一個「數位員工」人設。

2. 改用 qmd skill 支援多集合與更好檢索 fast

大致流程：

```
bash

# 安裝/啟用 qmd skill (依官方文件)
# 新增集合
qmd collection add ~/kb_product_support --name product
qmd collection add ~/kb_legal_consult --name legal

# 產生嵌入
qmd embed

# 在 Hermes 設定中使 qmd 作為 MCP 工具
```

3. 為每個領域建立不

詢問後續問題

+

搜尋

電腦

模型

↓

↑

- ☐ Hermes 已安裝且可正常對話。
- ☐ `pip install chromadb` 成功。
- ☐ `hermes config set knowledge_base_provider chromadb` 成功。
- ☐ 已建立知識庫資料夾 (如 `~/kb_product_support`) 並放入文件。
- ☐ 使用 `hermes docs add` 將文件加入知識庫。
- ☐ 確認 VM 可連到 Windows 上的 LM Studio API。
- ☐ 在 Hermes 設定好 OpenAI-compatible provider 指向 LM Studio。
- ☐ 設定好系統提示詞, 限制只能依知識庫回答。
- ☐ 用 `hermes chat` 測試問答, 確認行為符合預期。

如果你願意, 我可以在下一步幫你:

- 依你實際的 Hermes 版本, 寫出「可直接複製貼上」的完整指令腳本 (含註解)。
- 或幫你設計一個「產品客服」與「法律諮詢」的系統提示詞範本, 直接套用到你的知識庫。

詢問後續問題

+

搜尋

電腦

模型

↓

↑

可以, 但 **llm-wiki** 不是單純的 RAG 替代品, 而是「預先編譯的知識層」。它把原始文件先由 LLM 整理成結構化、互相連結的 Markdown wiki; 查詢時可對 wiki 做檢索, 而不必每次都對原始 PDF/文件做向量檢索。

對你的「產品客服 / 法律諮詢數位員工」來說, 最穩的做法是: **llm-wiki** 作為主要知識庫, **RAG** 作為原始文件兜底檢索, 兩者搭配, 而不是完全捨棄 RAG。

兩者定位差異

| 面向   | 傳統 RAG             | llm-wiki                  |
|------|--------------------|---------------------------|
| 知識形態 | 原始文件切塊 + 向量索引      | LLM 整理後的結構化 Markdown wiki |
| 查詢方式 | 每次提問都檢索原始片段        | 先查已編譯的 wiki, 必要時再回查原始文件   |
| 適合資料 | 手冊、FAQ、法條、大量非結構化文件 | 高頻問題、流程、定義、政策、需要交叉引用的知識   |
| 維護成本 | 新文件加入後重新索引即可       | 新知識需經 LLM 編譯、審核與更新        |
| 風險   | 檢索片段可能不完整、答案較零散    | 編譯錯誤會被固化, 需要人工審核          |
| 適合場景 | 知識量大、更新頻繁          | 知識相對穩定、重複問題多              |

llm-wiki 的核心是「知識層」, 它會先將原始文件重新檢索; 因此它特別適合處理 SOP、產品規格、常見法律問題這類會反覆被問到的內容。

詢問後續問題

+

搜尋

電腦

模型

↓

↑

使用者提問

↓

Hermes Agent

- 1. 先查 llm-wiki (高品質、結構化答案)
- 2. 未命中或信心不足 → 查 RAG 向量索引 (原始文件兜底)
- 3. 產生回答, 並標註來源

這種「Wiki 優先、RAG 兜底」的模式, 也正好對應社群中常見的雙引擎設計: **wiki** 處理已沉澱的優質答案, RAG 處理尚未整理進 wiki 的原始資料。

在 VM 內的實作步驟

1. 建立兩個知識層

bash

```
bash

mkdir -p ~/digital_employee/{raw_docs,wiki,rag_index}
```

- `raw_docs/`：放原始 PDF、DOCX、TXT、Markdown。
- `wiki/`：放 llm-wiki 產生的結構化 Markdc。
- `rag_index/`：放 RAG 索引。

Pro 已啟用進階搜尋的免費預覽。

瞭解詳情

×

## 2. 先建立 RAG 兜底層

詢問後續問題

+

搜尋

電腦

模型

↓

↑

```
hermes config set knowledge_base_path ~/digital_employee/rag_index
hermes docs add ~/digital_employee/raw_docs --name product_support_raw
```

這一層確保：即使某個問題還沒被整理進 `wiki`，系統仍能從原始手冊或法條中檢索到依據。

## 3. 建立 llm-wiki 編譯層

llm-wiki 有多種實作，選擇時建議挑選：

- 支援本地 LLM 或 OpenAI-compatible API 的版本。
- 可輸出 Markdown、`llms.txt`、JSON-LD 或可供 Agent 查詢格式的版本。
- 支援來源引用，避免 LLM 編譯時遺失原始依據。

一般流程是：

```
bash

# 示意流程，實際指令依你選的 llm-wiki 實作而定
llm-wiki init ~/digital_employee/wiki
llm-wiki ingest ~/digital_employee/raw_docs/product_manual.pdf
llm-wiki ingest ~/digital_employee/raw_docs/s/faq.md
llm-wiki compile
```

Pro 已啟用進階搜尋的免費預覽。

瞭解詳情

×

它會把原始素材轉成：

詢問後續問題

+

搜尋

電腦

模型

↓

↑

- 交叉引用，例如 `[[保固政策]]`、`[[退貨流程]]`。

llm-wiki 的設計正是把零散素材整理成持久、互相連結的 `wiki`，而不是每次查詢都重新檢索原始文件。

## 4. 讓 Hermes 讀取 wiki

有兩種接法：

方法 A：把 `wiki` 當成 Hermes 的第二個文件知識庫

```
bash

hermes docs add ~/digital_employee/wiki --name product_support_wiki
```

優點：設定最簡單，Hermes 會把 `wiki` 頁面納入檢索。

方法 B：讓 Hermes 用工具查詢 `wiki`

若你的 llm-wiki 提供 CLI 或 MCP 工具，可在 Hermes 的 `skill / tool` 設定中加入：

text

↓

工具名稱：`query_llm_wiki`

用途：查詢已編譯的 `wiki` 內容。

輸入：使用者問題

Pro 已啟用進階搜尋的免費預覽。

瞭解詳情

×

詢問後續問題

+

搜尋

電腦

模型

↓

↑

## 5. 修改系統提示詞

將原本「只能依知識庫回答」改成明確的雙層規則：

text

你是一個「產品客服專家」。

回答規則：

1. 優先查詢 `llm-wiki` 知識庫；若找到明確、完整的答案，依 `wiki` 內容回答。
2. 若 `wiki` 沒有足夠資訊，再查詢 `RAG` 原始文件索引。
3. 只能根據 `wiki` 或 `RAG` 檢索到的內容回答。
4. 若兩者都沒有可靠依據，必須回答：「目前知識庫中沒有足夠資料，無法確認。」
5. 不得推測、編造政策、保固條款或法律結論。
6. 回答時請標示資料來源，例如來自 `wiki` 頁面名稱或原始文件名稱。

這能避免 LLM 把 `wiki` 的摘要當成唯一事實，也能在 `wiki` 未覆蓋時回到原始文件驗證。

## 何時適合用 llm-wiki 取代 RAG

可以幾乎取代 `RAG` 的情況：

- 知識庫不大，例如數本知識文件。
- 內容以 SOP、FAQ

Pro 已啟用進階搜尋的免費預覽。 瞭解詳情 ×

詢問後續問題

+ 🔍 搜尋 🖨 電腦

模型 下載 上傳

適用於 RAG 的提示。

- 文件量大、更新快。
- 有大量合約、判例、技術手冊等長篇原始文件。
- 需要對原始條文做精確引用。
- 法律、醫療、金融等高風險領域，必須能回查原始依據。

### 建議你的落地版本

以你的環境，我會建議：

1. 第一階段：Hermes + ChromaDB RAG，先把原始產品手冊 / FAQ / 法條建好索引。
2. 第二階段：用 llm-wiki 將高頻知識編譯成 `product_support_wiki`。
3. 第三階段：Hermes 改成「wiki 優先、RAG 兜底」。
4. 第四階段：每個領域一個 wiki + 一個 RAG collection，形成可擴充的多數位員工。

這樣你不需要在「RAG 或 llm-wiki」之間二選一；llm-wiki 負責讓答案更穩定、結構化，RAG 負責保證可回溯到原始文件。

github +1

📄 📌 📄 📄 📄 📄 16 個來源 ...

↳ 看看 llm-wiki 的實際編譯指令與檔案結構

↳ 要在 Linux VM 安裝哪一個 llm-wiki 開源專案

↳ 在 Hermes Agent 中設定 wiki 優先的查詢順序

↳ 評估 llm-wiki 在 128GB 本地硬體上的編譯效能

↳ 如何讓多個數位員工共用同一個 llm-wiki

Pro 已啟用進階搜尋的免費預覽。 瞭解詳情 ×

詢問後續問題

+ 🔍 搜尋 🖨 電腦

模型 下載 上傳